

1. 环境准备

- 克隆 LLaMA-Factory 仓库并配置环境

2. 准备数据集

3. 启动llama-factory的webui

4. 从HuggingFace上下载大模型到本地进行微调

5. 下载数据集

6. 开始训练

选择模型

训练设置

预览命令

开始训练

训练结束

训练结果(命令行查看)

7. 将训练好的模型转换GGUF格式

(1) 合并非全量微调的LORA模型和原来的模型

(2) 使用llama.cpp将合并后的文件转换为ollama可以本地部署的GGUF格式

克隆git仓库代码到本地

Conda虚拟环境

安装依赖

运行convert_hf_to_gguf.py文件转换格式

(3) 创建本地ollama模型

新建Modelfile文件

使用ollama创建模型

命令行运行模型

openwebui前端使用

其他内容

1. 留痕软件 把微信聊天记录制作成电子书 (没尝试)

2. Hugging Face 下载模型的三种方式

huggingface-cli

snapshot_download (Python脚本) [没尝试]

git lfs (大模型推荐) [没尝试]

3. 登录huggingface

4. 验证torch和cuda是否安装成功

5. wget 下载

6. 参考教程

使用 LLaMA-Factory 训练自定义大模型并支持 Ollama 接入，需按以下步骤：

1. 环境准备

- 克隆 LLaMA-Factory 仓库并配置环境

克隆代码到本地安装python包

```
git clone https://github.com/hiyouga/LLaMA-Factory.git
```

```
#打开文件夹  
cd LLaMA-Factory
```

Conda虚拟环境

```
conda create -n llamaf11 Python=3.11
conda activate llamaf11
```

安装依赖（推荐 Python 3.10+）

```
# 我安装的3.11版本，创建了llamaf11的conda虚拟环境
pip install -r requirements.txt
pip install -e ".[torch,metrics]"
```

tips: 以下的CUDA和量化 BitsAndBytes 安装是看参考教程1的，不安装代码也可以运行

安装CUDA

查看显卡对应的cuda版本: <https://pytorch.org/get-started/locally/>

```
#执行下述代码下载cuda
pip3 install torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/cu126
```

量化 BitsAndBytes 安装

```
pip install bitsandbytes
```

2. 准备数据集

参考bigmodel-datasets中的详解，跳转到[readme_zn.md](#)

1. **格式**: JSON 文件（支持 Alpaca/ShareGPT ）格式

```
Alpaca格式
[
{
"instruction": "解释人工智能",
"input": "",
"output": "人工智能是模拟人类智能的计算机系统..."
},
...
]
```

2. **放置路径**: 创建 data/your_data.json , 在 dataset_info.json 中添加配置

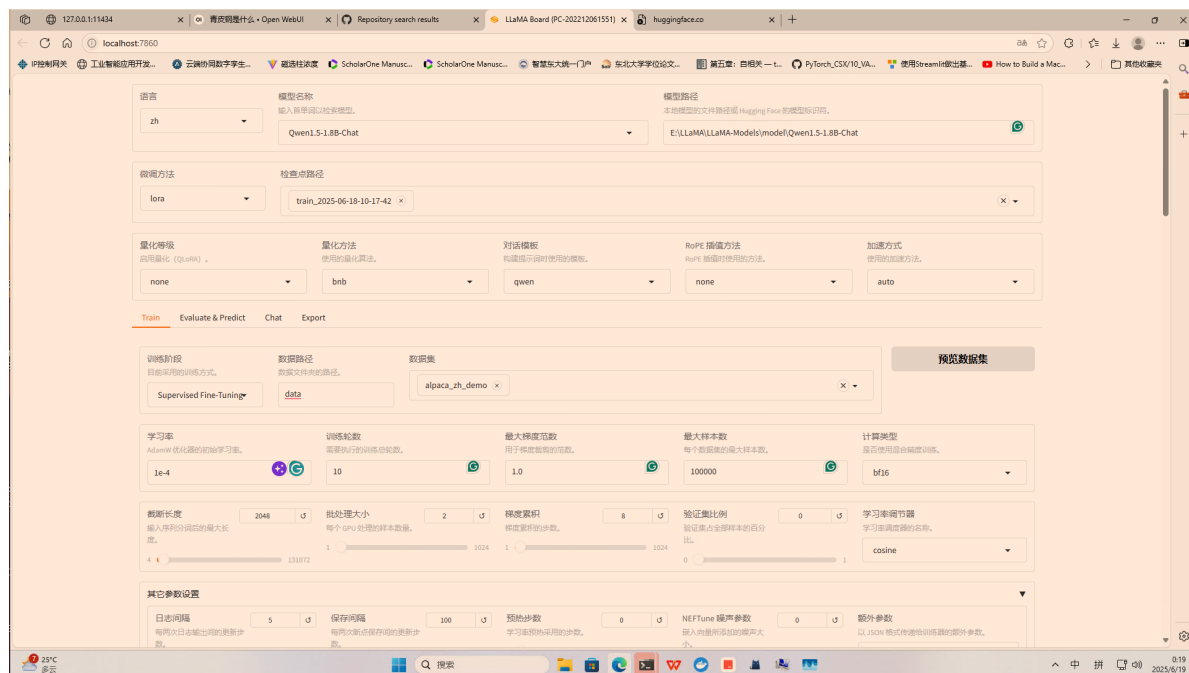
```
"your_dataset": {
  "file_name": "your_data.json",
  "file_sha1": "" // 留空跳过校验
}
```

tips: 这里我训练用的是自带的Alpaca格式的json数据集alpaca_zn_demo.json

3. 启动llama-factory的webui

llamafactory-cli webui

运行起来之后访问：<http://localhost:7860/>即可看到运行的交互界面



4. 从HuggingFace上下载大模型到本地进行微调

[Models - Hugging Face](#)

使用hugging cli方式，需要登录hugging，这里我选择Qwen1.5-1.8B-Chat模型，根据不同的模型输入，可以选择不同的模型

```
pip install huggingface-hub
huggingface-cli download Qwen/Qwen1.5-1.8B-Chat --local-dir ./model/Qwen1.5-1.8B-Chat
```

5. 下载数据集

暂时没找到好用的开源模型，但是可以参考以下地址下载模型：

[Hugging Face - The AI community building the future.](#)

[数据平台](#)

6. 开始训练

选择模型

语言→模型名称→模型路径

语言zh

模型名称Qwen1.5-1.8B-Chat

模型路径E:\LLaMA\LLaMA-Models\model\Qwen1.5-1.8B-Chat

微调方法lora

检查点路径train_2025-06-18-10-17-42

量化等级none

量化方法bnb

对话模板qwen

RoPE 插值方法none

加速方式auto

Train

Evaluate & Predict

Chat

Export

训练阶段Supervised Fine-Tuning

数据路径data

数据集alpaca_zh_demo

预览数据集

学习率1e-4

训练轮数10

最大梯度范数1.0

最大样本数100000

计算类型bf16

截断长度2048

批处理大小2

梯度累积8

验证集比例0

学习率调节器cosine

其它参数设置

日志间隔5

保存间隔100

预热步数0

NEFTune 噪声参数0

额外参数["optim", "adamw_torch"]

训练设置

数据集→预览数据集→学习率→训练轮数→最大样本数→梯度

微调方法lora

检查点路径

量化等级none

量化方法bnb

对话模板qwen

RoPE 插值方法none

加速方式auto

Train

Evaluate & Predict

Chat

Export

训练阶段Supervised Fine-Tuning

数据路径data

数据集alpaca_zh_demo

预览数据集

学习率1e-4

训练轮数10

最大梯度范数1.0

最大样本数100000

计算类型bf16

截断长度2048

批处理大小2

梯度累积8

验证集比例0

学习率调节器cosine

其它参数设置

日志间隔5

保存间隔100

预热步数0

NEFTune 噪声参数0

额外参数["optim", "adamw_torch"]

预览命令

The screenshot displays the LLaMA-Factory web interface. At the top, there are four tabs: "预览命令" (Preview Command), "保存训练参数" (Save Training Parameters), "载入训练参数" (Load Training Parameters), and "开始" (Start). The "预览命令" tab is active, showing a terminal window with the command: `llamafactory-cli train --stage sft --do_train True --model_name_or_path E:\llama\LLaMA-Models\model\Qwen1.5-1.8B-Chat --preprocessing_num_workers 16 --finetuning_type lora --template qwen --flash_attn auto --dataset_dir data --dataset ningsigang_alpaca_zh_demo --cutoff_len 8040 --learning_rate 0.0001 --num_train_epochs 10.0 --max_samples 100000 --per_device_train_batch_size 2 --gradient_accumulation_steps 8 --lr_scheduler_type cosine --max_grad_norm 1.0 --logging_steps 5 --save_steps 100 --warmup_steps 0 --packing False --enable_thinking True --report_to none --output_dir saves\Qwen1.5-1.8B-Chat\lora\train_2025-06-18-17-42 --bf16 True --plot_loss True --speed_monitor off --save`. A red arrow points from the "预览命令" tab to the command preview area. To the right of the command preview, there are sections for "输出目录" (Output Directory) and "配置路径" (Config Path), both set to "train_2025-06-18-17-42". Below these, there are sections for "设备数量" (Number of Devices) set to 1, "DeepSpeed stage" set to none, and a checkbox for "使用 DeepSpeed offload" (Use DeepSpeed offload) which is unchecked. On the far right, there is a "训练损失" (Training Loss) graph showing a decreasing trend over 600 steps, with a legend for "engine" and "smoothed".

开始训练

预览命令

保存训练参数

载入训练参数

开始

中断

输出目录

保存结果的路径。

train_2025-06-18-10-17-42

配置路径

保存训练参数的配置文件路径。

2025-06-18-10-17-42.yaml

设备数量

当前可用的计算设备数。

1

DeepSpeed stage

多卡训练的 DeepSpeed stage。

none

使用 DeepSpeed offload (会减慢速度)。

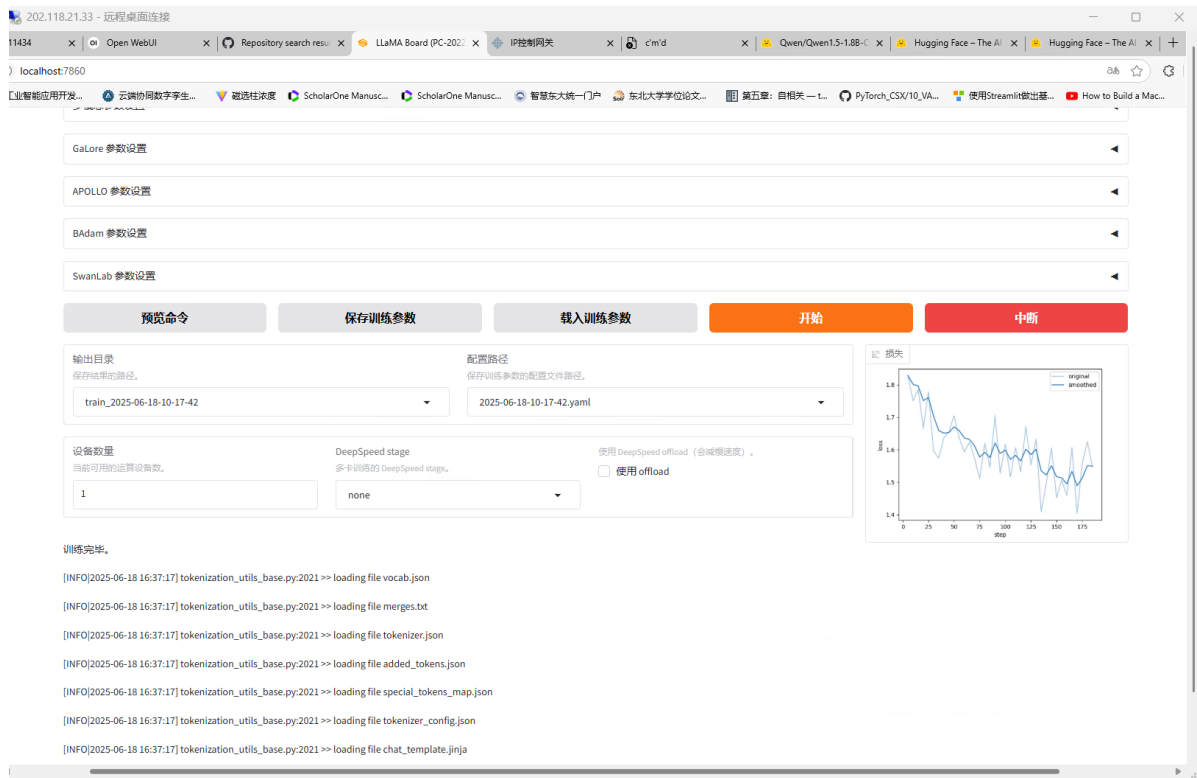
☐ 使用 offload

llamafactory-cli train`

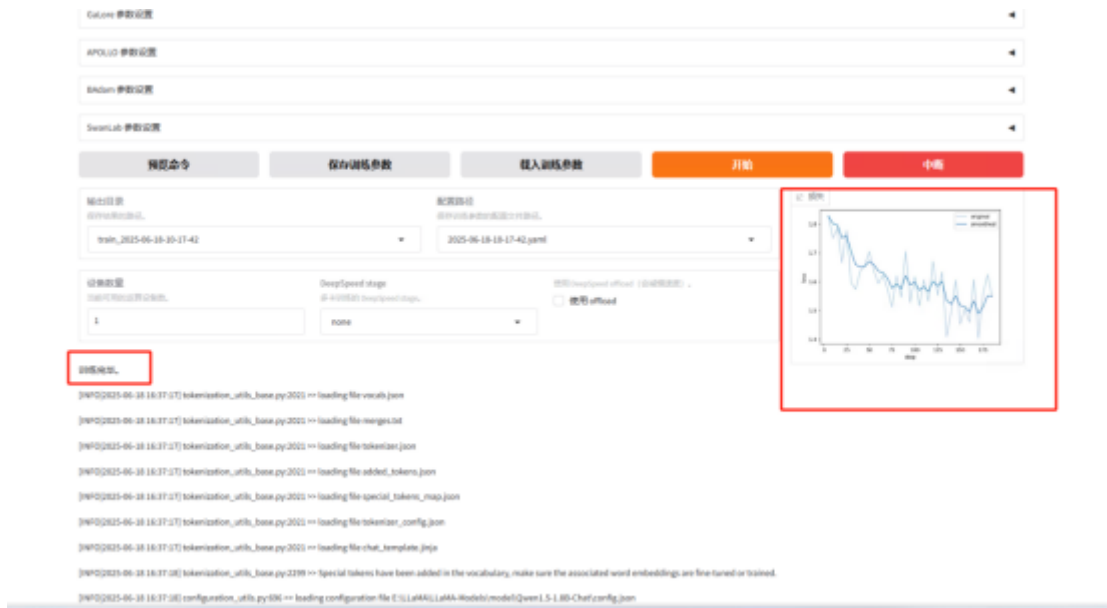
```
--stage sft`
--do_train True`
--model_name_or_path E:\LLaMA\LLaMA-Models\model\Qwen1.5-1.8B-Chat`
--preprocessing_num_workers 16`
--finetuning_type lora`
--template qwen`
--flash_attn auto`
--dataset_dir data`
--dataset qingpigang_alpaca_zh_demo`
--cutoff_len 2048`
--learning_rate 0.0001`
--num_train_epochs 10.0`
--max_samples 100000`
--per_device_train_batch_size 2`
--gradient_accumulation_steps 8`
--lr_scheduler_type cosine`
--max_grad_norm 1.0`
--logging_steps 5`
--save_steps 100`
--warmup_steps 0`
--packing False`
--enable_thinking True`
--report_to none`
--output_dir saves/Qwen1.5-1.8B-Chat\lora\train_2025-06-18-10-17-42`
--bf16 True`
```

损失

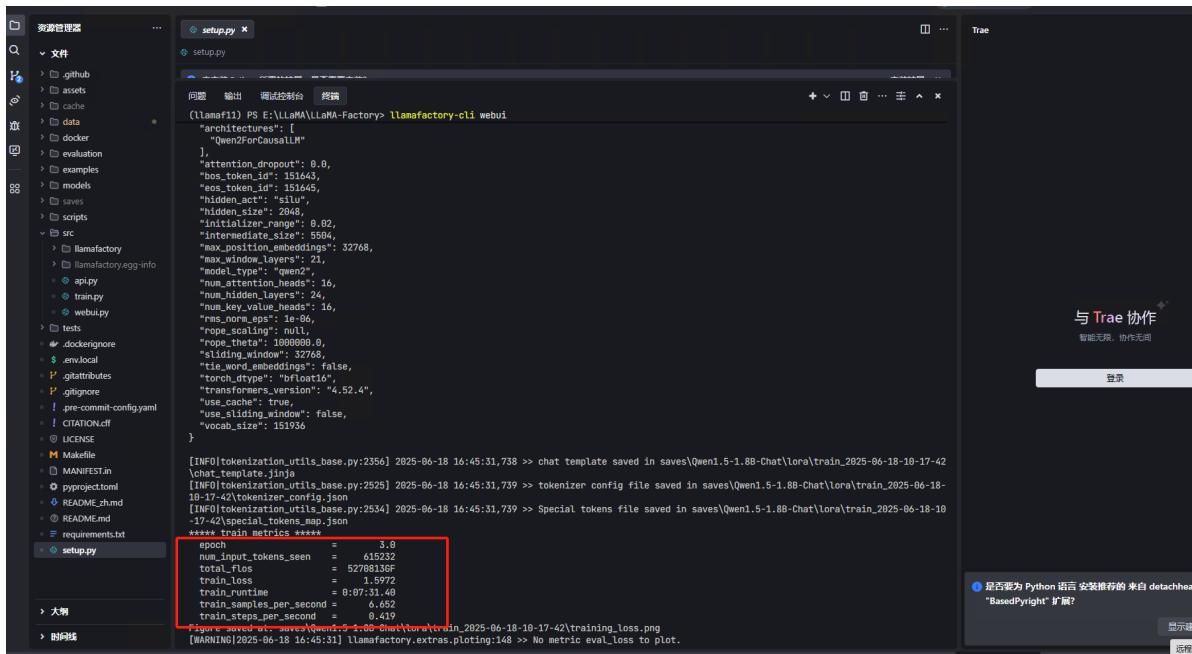
简体中文/中国大模型阵营



训练结束



训练结果(命令行查看)



epoch: 表示训练过程中数据集被完整地通过模型一次的次数。

num_input_tokens_seen: 表示在训练过程中模型已经看到的输入标记（例如单词或字）的总数。

total_flos: 表示训练过程中执行的浮点运算次数，单位是 GF。

train_loss: 表示训练集上的损失函数值，损失越低通常意味着模型性能越好。

train_runtime: 表示训练过程的总运行时间。

train_samples_per_second: 表示模型每秒可以处理的样本数。

train_steps_per_second: 表示模型每秒可以执行的训练步骤数。

7.将训练好的模型转换GGUF格式

(1) 合并非全量微调的LORA模型和原来的模型

新建一个文件夹**models**，创建一个**export_model.py**文件，用于将非全量微调的模型LORA和原来的模型合并，运行完成后llama-factory下面会新建一个文件夹**merged_model**存储合并后的模型

```
from transformers import AutoModelForCausalLM, AutoTokenizer
from peft import PeftModel
import os
# 定义模型路径
base_model_path = "E:/LLaMA/LLaMA-Models/model/Qwen1.5-1.8B-Chat"
lora_adapter_path = "./saves/Qwen1.5-1.8B-Chat/lora/train_2025-06-18-10-17-42"
output_path = "E:/LLaMA/LLaMA-Factory/merged_model"
# 如果merge不存在,则创建文件夹
os.makedirs(output_path, exist_ok=True)

# 加载基础模型和LORA适配器
print("正在加载基础模型...")
base_model = AutoModelForCausalLM.from_pretrained(
    base_model_path,
    trust_remote_code=True,
    device_map="auto"
)

print("正在加载 LORA 适配器...")
lora_model = PeftModel.from_pretrained(
    base_model,
```

```

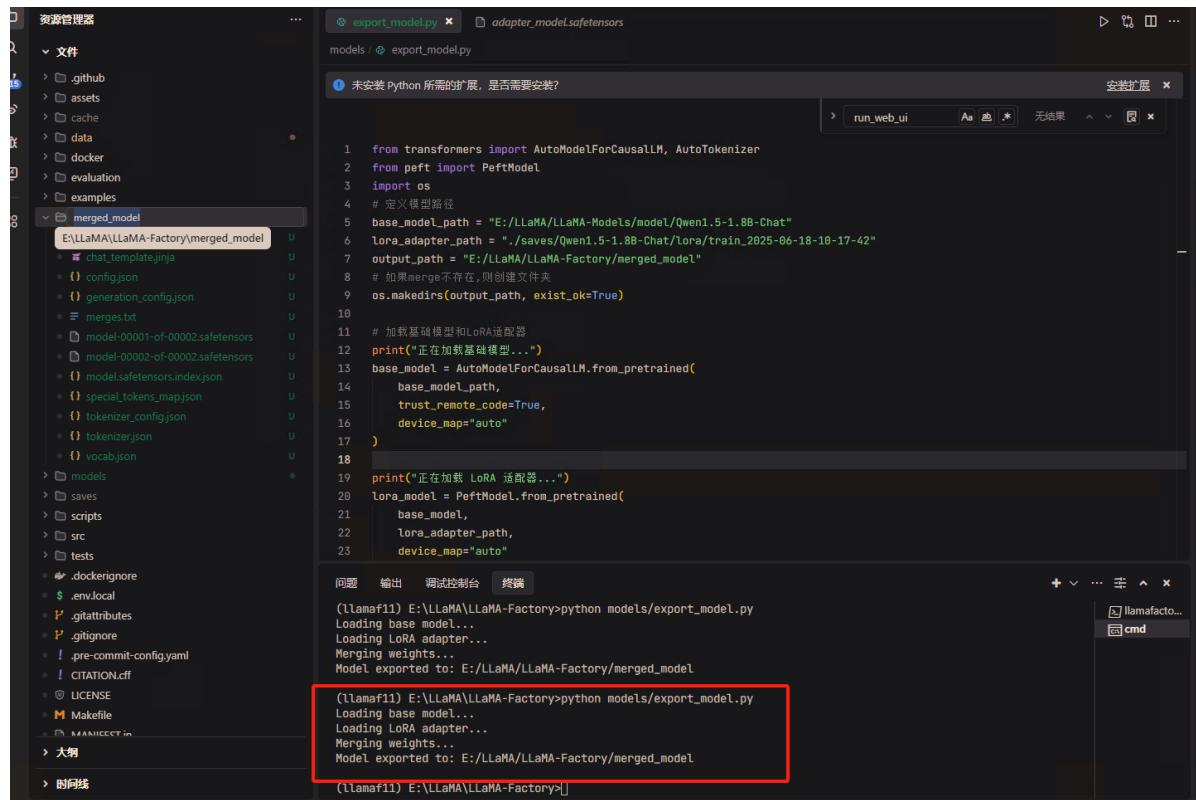
    lora_adapter_path,
    device_map="auto"
)
# 合并权重
print("正在合并模型...")
merged_model = lora_model.merge_and_unload()

# 保存合并后的完整模型
merged_model.save_pretrained(output_path, safe_serialization=True)
tokenizer = AutoTokenizer.from_pretrained(base_model_path)
tokenizer.save_pretrained(output_path)
print("模型合并完成! 保存路径:", output_path)

```

运行上述python文件

```
python models/export_model.py
```



(2) 使用llama.cpp将合并后的文件转换为ollama可以本地部署的GGUF格式

克隆git仓库代码到本地

```
git clone https://github.com/ggml-org/llama.cpp.git
```

Conda虚拟环境

```
conda create -n llamacpp11 Python=3.11
conda activate llamacpp11
```


安装依赖

```
pip install -r requirements.t
```

运行convert_hf_to_gguf.py文件转换格式

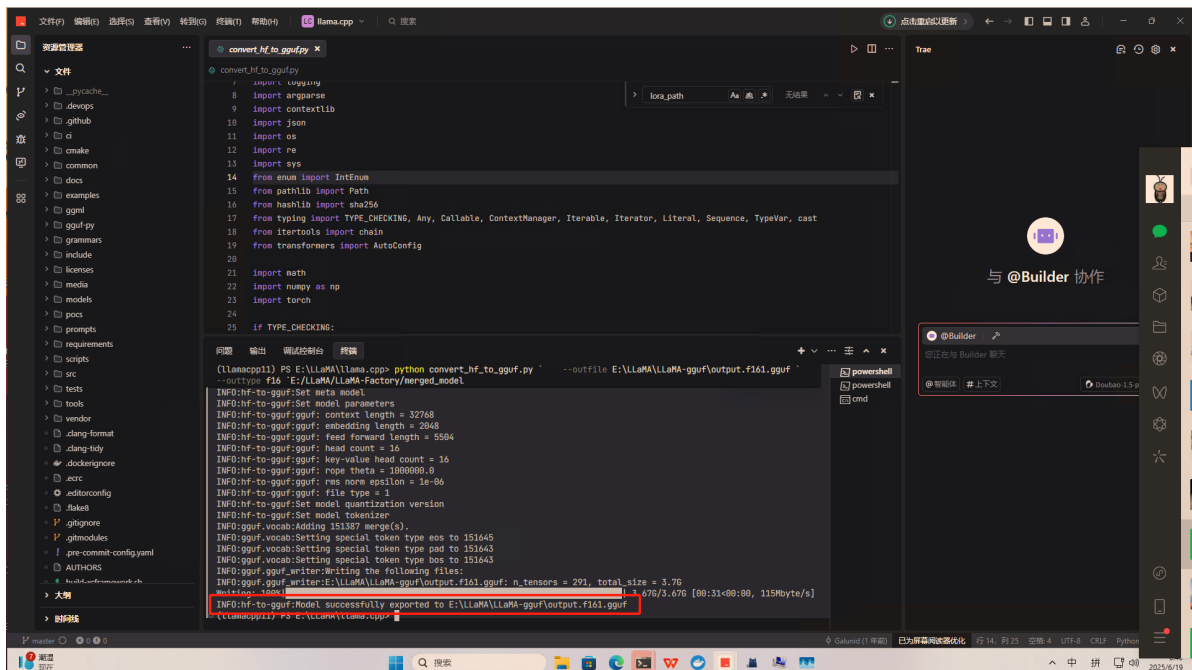
打开完成后看到本地有4个convert.py文件，合并后的模型选用**convert_hf_to_gguf.py**

```
python convert_hf_to_gguf.py `
    --outfile E:\LLaMA\LLaMA-gguf\output.f161.gguf `
    --outtype f16 `
    E:/LLaMA/LLaMA-Factory/merged_model
```

convert_lora_to_gguf.py是未合并的 LoRA 适配器

convert_llama_ggml_to_gguf.py是旧版 GGML 模型

其中 E:/LLaMA/LLaMA-Factory/merged_model是上一步合并模型的位置，outfile是对应的输出位置和文件名称 E:\LLaMA\LLaMA-gguf\output.f161.gguf，结果如下



(3) 创建本地ollama模型

新建Modelfile文件

```
#sharegpt格式Modelfile
FROM E:\LLaMA\LLaMA-gguf\output.f161.gguf

TEMPLATE """<|im_start|>system
{{ .System }}<|im_end|>
<|im_start|>user
{{ .Prompt }}<|im_end|>
<|im_start|>assistant
"""

PARAMETER num_ctx 4096
PARAMETER stop "<|im_end|>"
PARAMETER stop "<|im_start|>"
```

```
PARAMETER temperature 0.7
```

```
#Alpaca格式 ModelfileA
FROM E:/LLaMA/LLaMA-gguf/output.f16.gguf

TEMPLATE """<|im_start|>system
你是一个AI助手<|im_end|>
<|im_start|>user
{{ .Prompt }}<|im_end|> # 此处合并 instruction 和 input
<|im_start|>assistant
"""

PARAMETER num_ctx 4096
PARAMETER stop "<|im_end|>"
PARAMETER stop "<|im_start|>"
PARAMETER temperature 0.7
```

使用ollama创建模型

在Modelfile文件夹下打开cmd，下述两个代码选一个创建模型

```
# 其中qw-15是模型名称，后面是部署的配置文件
ollama create qw-15 -f .\ModelfileA

ollama create qw-15 -f .\Modelfile
```

命令行运行模型

```
ollama run qw-15
```

openwebui前端使用

<http://localhost:3000/>



其他内容

1.留痕软件 把微信了聊天记录制作成电子书（没尝试）

2.Hugging Face下载模型的三种方式

huggingface-cli

```
pip install huggingface-hub

huggingface-cli download Qwen/Qwen-1.8B-Chat --local-dir ./model/Qwen-1.8B-Chat
```

snapshot_download (Python脚本) [没尝试]

```
from huggingface_hub import snapshot_download

snapshot_download(

    repo_id="THUDM/chatglm3-6b",

    local_dir="./model/chatglm3-6b",

    resume_download=True

)
```

git lfs (大模型推荐) [没尝试]

```
git lfs install

git clone https://huggingface.co/OpenBuddy/openbuddy-llama2-13b-v8.1-fp16
./model/openbuddy-13b
```

3.登录huggingface

```
huggingface-cli login
```

要获取自己的token

4.验证torch和cuda是否安装成功

```
import torch

torch.cuda.current_device()

torch.cuda.get_device_name(0)

torch.__version__
```

```
管理品: Anaconda Prompt (An x + v - □ x)

(base) C:\Users\Administrator>conda activate llamaf11

(llamaf11) C:\Users\Administrator>python
Python 3.11.13 | packaged by Anaconda, Inc. | (main, Jun 5 2025, 13:03:15) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import torch
>>> torch.cuda.current_device()
0
>>> torch.cuda.get_device_name(0)
'NVIDIA GeForce RTX 4090'
>>> torch.__version__
'2.7.1+cu126'
>>> |
```

5.wget下载

[Tool之wget: wget的简介、安装、入门、使用方法之详细攻略（一）-阿里云开发者社区](#)

6.参考教程

1. [LLaMA-Factory DeepSeek-R1 模型 微调基础教程 llamafactory微调deepseek-CSDN博客](#)

[T2D: Generating Dialogues Between Virtual Agents Automatically from Text | SpringerLink](#)

个性化对话生成的最新趋势：数据集、方法和评估综述

[论文阅读七：LLaMA-Factory: 100多种语言模型的统一高效微调 | Model The World](#)